Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

F. Gợi ý hướng giải 2
F.1. Chương 1: Từ monolith đến microservices 2
F.2. Chương 2: DDD & Bounded Contexts 2
F.3. Chương 3: Thiết kế API 3
F.4. Chương 4: Giao tiếp đồng bộ & Resilience 3
F.5. Chương 5: Giao tiếp bất đồng bộ 4
F.6. Chương 6: Saga Pattern 4
F.7. Chương 7: Quản lý dữ liệu 4
F.8. Chương 8: API Gateway 5
F.9. Chương 9: Bảo mật 5
F.10. Chương 10: Chuyển đổi thực tế 5
F.11. Chương 11: Observability 6
F.12. Chương 12: Triển khai & DevOps 6
F.13. Capstone: Thiết kế “Online Judge” 6

PHỤ LỤC F

Gợi ý hướng giải

Gợi ý hướng giải cho các bài tập trong phần "Bài tập & case study". Gợi ý chỉ định hướng, không phải đáp án đầy đủ: nhiều bài (đặc biệt mức `L2` và `L3`) là bài mở, không có một đáp án đúng duy nhất; giá trị nằm ở quá trình phân tích và lập luận.

F.1. Chương 1: Từ monolith đến microservices

1-0 `NOTE` **Ôn tập** `L1`: Các đáp án tương ứng là A là 2, B là 1, C là 2, D là 3, E là 1, F là 2. Martin Fowler khuyên nên bắt đầu bằng monolith (Monolith First), chỉ tách nhỏ khi có đủ lý do xác đáng. Xem §1.6. Đối với khả năng mở rộng, kiến trúc monolith bắt buộc phải nhân bản toàn bộ hệ thống ngay cả khi chỉ một tính năng bị quá tải; trong khi kiến trúc Microservices cho phép chỉ nhân bản dịch vụ chịu tải cao, giúp phân bổ tài nguyên hiệu quả hơn.

1-1 `EVAL` `L2`: Công ty A: chưa nên (Not Yet) do đang là startup, chưa đạt độ phù hợp sản phẩm-thị trường (product-market fit). Công ty B: nên chuyển đổi (Yes) vì đã có đủ đội ngũ và điểm nghẽn (pain point) rõ ràng. Công ty C: không bao giờ hoặc chưa nên (Never/Not Yet) vì đây chỉ là hệ thống ERP nội bộ với lượng người dùng rất nhỏ (500 người dùng). Xem Bảng 10.1 về ma trận quyết định (Decision Matrix).

1-2 `CASE` `L3`: Shopify lựa chọn kiến trúc Modular Monolith vì ranh giới giữa các thành phần (component boundaries) đã đáp ứng đủ cho mức độ mở rộng (scale) của họ. Xem §1.6.

F.2. Chương 2: DDD & Bounded Contexts

2-0 `NOTE` **Ôn tập** `L1`: Bounded Context tương ứng với (b); Ubiquitous Language tương ứng với (b); Aggregate tương ứng với (a). Thực thể "Bài nộp" thuộc ngữ cảnh SQL Practice; thực thể "Tài khoản" thuộc ngữ cảnh Identity & Access. Các hệ thống giao tiếp thông qua API hoặc sự kiện. Chuyên gia nghiệp vụ (Domain Expert) đảm bảo mô hình phần mềm phản ánh chính xác quy trình thực tế; nếu kỹ sư tự thiết kế có thể dẫn đến sai lệch ranh giới ngữ cảnh và ngôn ngữ chung (Ubiquitous Language).

2-1 `DESIGN` `L2`: Luồng sự kiện: tìm kiếm môn học (CourseSearched), kiểm tra điều kiện tiên quyết (PrerequisiteChecked), yêu cầu đăng ký (RegistrationRequested), giữ chỗ thành công (SeatReserved) hoặc vào danh sách chờ (WaitlistJoined), khởi tạo thanh toán (BillingInitiated), thanh toán hoàn tất (PaymentCompleted) hoặc hết hạn thanh toán (PaymentExpired), xác nhận đăng ký (RegistrationConfirmed) hoặc hủy đăng ký (RegistrationCancelled). Các ngữ cảnh gồm: danh mục môn học, đăng ký, thanh toán và hồ sơ sinh viên.

2-2 `EVAL` `L2`: Logic kiểm tra tính hợp lệ (Validation logic) thuộc về nghiệp vụ (domain logic) nên cần được giữ lại bên trong dịch vụ, không chia sẻ ra ngoài. Thư viện dùng chung (Shared library) chỉ nên chứa các đối tượng chuyển giao (DTO), mã lỗi (error code) và các chức năng cắt ngang (cross-cutting) như ghi nhật ký (logging). Xem Newman [4a].

F.3. Chương 3: Thiết kế API

3-0 `NOTE` **Ôn tập** `L1`: A là non-breaking (thêm trường dữ liệu); B là breaking (đổi tên đồng nghĩa với xóa trường cũ và thêm trường mới); C là non-breaking (tham số tùy chọn); D là breaking (thay đổi mã trạng thái HTTP); E là breaking (xóa trường); F là breaking (thêm trường bắt buộc). Bộ tiêu thụ khoan dung (Tolerant Reader) là cơ chế mà dịch vụ tiêu thụ tự động bỏ qua các trường dữ liệu mà nó không nhận diện được. Xem §3.9. Tính tương thích ngược đặc biệt trọng yếu trong Microservices vì các dịch vụ được triển khai độc lập; phá vỡ hợp đồng dữ liệu sẽ gây lỗi trực tiếp cho các dịch vụ tiêu thụ chưa được cập nhật.

3-1 `CASE` `L2`: Stripe sử dụng định dạng phiên bản theo ngày (date-based versioning) để đảm bảo tính tương thích ngược dài hạn (backward compatibility). Idempotency Key là một chuỗi định danh (UUID) do client sinh ra và đính kèm vào header của request. Xem §3.1 (Idempotency Key), §3.2 (versioning).

3-2 `DEBUG` `L3`: Nguyên nhân gốc rễ (Root cause): việc đổi tên trường dữ liệu (tương đương xóa trường cũ và thêm trường mới) khiến các bộ tiêu thụ phiên bản cũ gặp lỗi. Biện pháp phòng tránh (Prevention): kiểm thử hợp đồng (contract testing) và áp dụng mẫu Expand-and-Contract (mở rộng rồi thu hẹp). Xem §3.2, §3.6.

F.4. Chương 4: Giao tiếp đồng bộ & Resilience

4-0 `NOTE` **Ôn tập** `L1`: A là 2, B là 4, C là 1, D là 5, E là 3. Circuit Breaker chuyển từ trạng thái Closed sang Open khi lỗi vượt ngưỡng, sau một khoảng thời gian thì sang Half-Open để thử lại, và quay lại Closed nếu thành công. Xem §4.4.

4-1 `DEBUG` `L3`: Nguyên nhân gốc rễ: thiếu Circuit Breaker và đặt timeout quá dài (30 giây). Bằng việc áp dụng Circuit Breaker, dịch vụ sẽ báo lỗi nhanh (fail-fast) thay vì chờ đợi timeout cạn kiệt. Kỹ thuật Bulkhead giúp giới hạn phạm vi ảnh hưởng (blast radius). Xem §4.4, §4.5.

4-2 `EVAL` `L2`: A chọn GraphQL vì phù hợp cho các truy vấn linh hoạt và băng thông hạn chế; B chọn gRPC vì đáp ứng thông lượng cao và lược đồ dữ liệu nghiêm ngặt; C chọn GraphQL vì phù hợp cho cấu trúc dữ liệu lồng nhau; D chọn REST để đảm bảo tính ổn định và dễ dàng lập tài liệu. Xem §4.1 (và §3.7 cho GraphQL).

F.5. Chương 5: Giao tiếp bất đồng bộ

5-0 `NOTE` **Ôn tập** `L1`: Các thành phần gồm Topic, Partition, Offset, Consumer Group, Producer và Message Broker. Luồng xử lý bài nộp (submission pipeline) của KBM nên áp dụng chiến lược chuyển giao ít nhất một lần (at-least-once) kết hợp với idempotent consumer. Xem §5.3, §5.5.

5-1 `EVAL` `L2`: A chọn RabbitMQ vì phù hợp cho định tuyến đơn giản và cơ chế xác nhận; B chọn Kafka vì đảm bảo thứ tự và lưu giữ luồng sự kiện; C chọn Kafka vì đáp ứng thông lượng cao; D chọn RabbitMQ vì điều phối vòng tròn và cơ chế xác nhận. Xem §5.2.

5-2 `DEBUG` `L3`: Thông điệp độc (Poison pill) là các thông điệp không thể giải tuần tự hóa (deserialize), dẫn đến việc bộ tiêu thụ bị lỗi lặp vô tận (crash loop). Giải pháp là tiến hành thử lại (retry) 3 lần, sau đó đẩy sang hàng đợi lỗi (DLT) và tiến hành rà soát thủ công (manual review). Xem §5.9.

F.6. Chương 6: Saga Pattern

6-0 `NOTE` **Ôn tập** `L1`: Thứ tự thực hiện: (1) Core tạo Submission, (2) Judge chấm bài, (3) Ghi điểm Gradebook, (4) Gửi thông báo. Compensation cho bước 1 là đánh dấu Submission là ERROR (C1 — Mark ERROR). Saga *không* đảm bảo ACID, nó đảm bảo ACD (tính nguyên tử qua compensation, tính nhất quán giữ bởi quy tắc nghiệp vụ trong mỗi local transaction, và tính bền vững — durability). Lưu ý: "nguyên tử qua compensation" là nguyên tử *nghiệp vụ* (hoàn tất toàn chuỗi hoặc bù trừ các bước đã làm), không phải rollback ACID — trạng thái trung gian vẫn có thể bị quan sát, và mỗi local transaction bên trong vẫn là ACID. Xem §6.2.

6-1 `DESIGN` `L3`: Ví dụ đặt xe Uber: Tạo chuyến (CreateRide), Ghép tài xế (MatchDriver), Thanh toán (ChargePayment), Xác nhận đón (ConfirmPickup). Orchestration phù hợp hơn vì luồng nghiệp vụ phức tạp và đòi hỏi nhiều kịch bản bù trừ (compensation). Xem §6.3.

6-2 `EVAL` `L2`: Mô hình Choreography đơn giản và có liên kết lỏng (loose coupling) nhưng khó truy vết (trace). Mô hình Orchestration cung cấp quản lý tập trung và dễ truy vết hơn, nhưng lại tạo ra điểm lỗi tập trung (single point of failure). Xem §6.3.

F.7. Chương 7: Quản lý dữ liệu

7-0 `NOTE` **Ôn tập** `L1`: A là 3, B là 2, C là 1, D là 4. Định lý CAP chỉ khi xảy ra tình trạng chia cắt mạng (Network Partition), hệ thống mới phải đánh đổi giữa tính nhất quán (Consistency) và tính khả dụng (Availability); trong điều kiện bình thường, hệ thống có thể đảm bảo cả C lẫn A. Vấn đề khi KBM dùng chung cơ sở dữ liệu là các dịch vụ bị trói buộc về lược đồ (schema coupling) và không thể mở rộng quy mô (scale) hoặc triển khai dịch vụ một cách độc lập. Xem §7.1.

7-1 `EVAL` `L3`: Khái niệm chọn 2 trong 3 của định lý CAP là sự hiểu lầm, vì khả năng chịu đựng chia cắt mạng (Partition tolerance — ký hiệu P) không phải là một tùy chọn bởi phân rã mạng luôn có thể xảy ra trong hệ thống phân tán. Chỉ khi sự phân rã thực sự xảy ra, hệ thống mới phải chọn giữa C hoặc A. Xem §7.1.

7-2 `DESIGN` `L2`: Bước 1 là tách schema (`core_schema`, `academic_schema`), truy vấn chéo còn lại tạm đọc qua view. Bước 2 là thay truy vấn chéo bằng API rồi thu hồi quyền đọc chéo. Bước 3 là nhân bản dữ liệu tham chiếu qua sự kiện. Tách hẳn database server chỉ khi cần mở rộng độc lập. Xem §7.2 (Áp dụng cho KBM).

F.8. Chương 8: API Gateway

8-0 `NOTE` **Ôn tập** `L1`: Thuộc Gateway gồm xác minh JWT ở biên (edge token validation / auth), Rate Limiting, Routing, SSL termination, Load balancing, và Response caching. Không thuộc Gateway gồm Business logic, Database migration, và Grading logic. Xem §8.4.

8-1 `CASE` `L2`: Netflix chuyển từ Zuul 1 (blocking, mỗi request một thread) sang Zuul 2 (non-blocking, dựa trên Netty) và vận hành Zuul 2 ở quy mô lớn. Spring Cloud Gateway là dự án riêng của Spring, xây trên WebFlux (non-blocking), thay thế Spring Cloud Netflix Zuul. Cái giá của non-blocking: luồng xử lý khó theo dõi, gỡ lỗi khó hơn, và mọi thành phần trên đường đi của request đều phải bất đồng bộ. Xem §8.2.

8-2 `ADR` `L2`: So sánh Token Bucket và Sliding Window. Thuật toán Token Bucket đơn giản hơn và cho phép xử lý các đợt lưu lượng tăng đột biến (burst). Trong khi đó, thuật toán Sliding Window mang lại độ chính xác cao hơn trong việc giới hạn thời gian. Xem §8.4.

F.9. Chương 9: Bảo mật

9-0 `NOTE` **Ôn tập** `L1`: A là AuthN; B là AuthZ; C là AuthN; D là AuthZ; E là AuthN; F là AuthZ. JWT gồm Header (algorithm), Payload (claims), và Signature. RS256 cho phép xác minh (verify) mà không cần bí mật chung (shared secret), điều này quan trọng khi có nhiều dịch vụ. Xem §9.2, §9.5.

9-1 `DEBUG` `L3`: Khi phát hiện token bị đánh cắp (Token theft), hệ thống cần áp dụng cơ chế danh sách đen (token blacklist), sử dụng access token có vòng đời ngắn (short-lived access token) kết hợp với cơ chế xoay vòng (rotation) cho refresh token. Xem §9.2.

9-2 `EVAL` `L2`: Luồng Authorization Code dành cho các ứng dụng có máy chủ (server-side app); Luồng Client Credentials dành cho giao tiếp nội bộ giữa các dịch vụ (service-to-service); Luồng Authorization Code kèm PKCE dành cho ứng dụng trang đơn (SPA) hoặc ứng dụng di động (mobile); Device Authorization Grant dành cho client không mở được trình duyệt (CLI, thiết bị hạn chế) nhưng vẫn cần đăng nhập với tư cách người dùng. Xem §9.4.

F.10. Chương 10: Chuyển đổi thực tế

10-0 `NOTE` **Ôn tập** `L1`: Thứ tự là B, C, E, A, D. Các anti-pattern gồm Big Bang rewrite (viết lại toàn bộ một lần) và Distributed Monolith. Chiến lược đập bỏ xây mới (Big Bang) là sai lầm do rủi ro quá cao; thay vào đó nên áp dụng mẫu thiết kế Strangler Fig để bóc tách từng phần. Đối với hệ thống KBM, Judge Service là ứng cử viên tốt nhất để tách trước vì nó có tính độc lập cao nhất và mang lại giá trị nghiệp vụ lớn. Xem §10.2, Phụ lục D.

10-1 `CASE` `L3`: Sự kiện API Mandate tại Amazon năm 2002 bắt buộc mọi nhóm phát triển phải bộc lộ chức năng thông qua API (expose service API). Quá trình chuyển đổi này thực tế đã kéo dài hơn 10 năm. Xem §10.1.

10-2 `EVAL` `L2`: Mẫu Outbox đơn giản và sử dụng giao dịch của cơ sở dữ liệu nội bộ; Mẫu Event Sourcing tạo ra lưu vết lịch sử (audit trail) nhưng có độ phức tạp cao hơn; Mô hình CDC can thiệp gián tiếp (không xâm lấn mã nguồn, non-invasive) nhưng yêu cầu công cụ bên thứ ba như Debezium. Xem §10.3.

10-3 `DESIGN` `L3`: (A) Khó cấu hình các trigger trên CSDL và làm ẩn đi logic nghiệp vụ; (B) Dễ lập trình nhưng tạo ra sự liên kết chặt chẽ (high coupling) vì monolith liên tục gọi đồng bộ đến API sẽ tạo ra điểm nghẽn; (C) Hàng đợi sự kiện (Event Queue) là phương án có liên kết lỏng nhất (loose

coupling) vì monolith chỉ cần lắng nghe sự kiện AttendanceRecorded để tự động cập nhật điểm. Rủi ro chung là việc bảo trì hai bộ mã nguồn, do đó cần nhanh chóng đóng băng hệ thống nguyên khối sớm nhất có thể.

F.11. Chương 11: Observability

11-0 `NOTE` **Ôn tập** `L1`: A là Metrics; B là Traces; C là Logs; D là Metrics; E là Traces; F là Logs. SLI là chỉ số đo lường thực tế (ví dụ độ trễ p99); SLO là mục tiêu nội bộ hệ thống cần đạt (ví dụ p99 dưới 500ms); SLA là bản cam kết pháp lý với khách hàng. Cảnh báo nên được thiết lập dựa trên SLO, vì nếu chỉ số phần cứng (CPU) cao nhưng trải nghiệm người dùng chưa bị ảnh hưởng thì chưa cần thiết phải báo động. Xem §11.1, §11.4.

11-1 `DEBUG` `L3`: Áp dụng truy vết phân tán (Distributed tracing) để xác định chính xác dịch vụ nào đang phản hồi chậm thông qua việc kiểm tra thời lượng (span duration) của từng bước xử lý. Xem §11.3.

11-2 `EVAL` `L2`: Tự xây dựng (Build) mang tính linh hoạt nhưng tiêu tốn công sức vận hành. Mua giải pháp (Buy) sử dụng các công cụ chuyên nghiệp như Datadog hoặc New Relic nhưng chi phí rất đắt đỏ. Dịch vụ đám mây (Managed) như AWS CloudWatch tiện lợi nhưng sẽ gặp rủi ro bị khóa chặt vào nhà cung cấp (vendor lock-in). Xem §11.1–11.3, §11.9.

F.12. Chương 12: Triển khai & DevOps

12-0 `NOTE` **Ôn tập** `L1`: A là 2 (Blue-Green), B là 1 (Canary), C là 4 (Rolling), D là 3 (Shadow). Container chia sẻ nhân hệ điều hành (OS kernel) nên khởi động nhanh; Máy ảo (VM) có hệ điều hành riêng, cung cấp khả năng cô lập mạnh mẽ hơn. Infrastructure as Code (IaC) đảm bảo mọi cấu hình đều được viết bằng mã, lưu trữ phiên bản và có tính lặp lại (reproducible). Đối với hệ thống KBM triển khai bằng Docker Compose đơn instance, thực chất chỉ có chiến lược Recreate (chấp nhận gián đoạn ngắn, chọn khung giờ thấp điểm) hoặc Blue-Green thủ công sau reverse proxy nếu cần triển khai không gián đoạn (zero-downtime); Rolling Update đúng nghĩa đòi hỏi orchestrator điều phối từng replica (Swarm/Kubernetes). Xem §12.2, §12.5.

12-1 `EVAL` `L2`: Docker Compose phù hợp cho môi trường phát triển, kiểm thử (staging) hoặc các hệ thống quy mô nhỏ. Kubernetes là lựa chọn hợp lý khi hệ thống yêu cầu tính năng tự động mở rộng (auto-scaling), khả năng tự phục hồi (self-healing) và sở hữu số lượng lớn dịch vụ. Xem §12.6.

12-2 `DESIGN` `L3`: Luồng triển khai tự động (Pipeline) bao gồm: gửi mã nguồn (commit), biên dịch (build), kiểm thử, triển khai môi trường staging, phê duyệt thủ công, phát hành canary lên production, và phát hành toàn bộ (full rollout). Phương pháp phát hiện dịch vụ bị thay đổi (Changed service detection) kết hợp kiểm tra khác biệt mã nguồn (git diff) và biểu đồ phụ thuộc (dependency graph). Xem §12.4.

F.13. Capstone: Thiết kế “Online Judge”

Capstone `L3`: Đề xuất 4 dịch vụ tối thiểu: API Gateway, Submission Service (nộp bài), Judge Service (chấm bài) và Leaderboard Service (bảng xếp hạng). Giao tiếp: sử dụng đồng bộ (REST hoặc gRPC) cho hành động nộp bài và bất đồng bộ (Kafka) cho hàng đợi chấm điểm. Dữ liệu: áp dụng kiến trúc CQRS cho bảng xếp hạng (sử dụng Redis Sorted Set cho mô hình đọc). Tính chịu lỗi (Resilience): đặt Circuit Breaker cho dịch vụ chấm bài, và hàng đợi lỗi (DLT) cho các bài nộp thất bại. Khả năng mở rộng (Scale): nhân bản theo chiều ngang (horizontal scale) dịch vụ chấm

bài, đồng thời chia partition Kafka theo `userId` để giữ thứ tự theo sinh viên (§5.9) — tránh key theo ID cuộc thi vì sẽ dồn cả kỳ thi vào một partition.